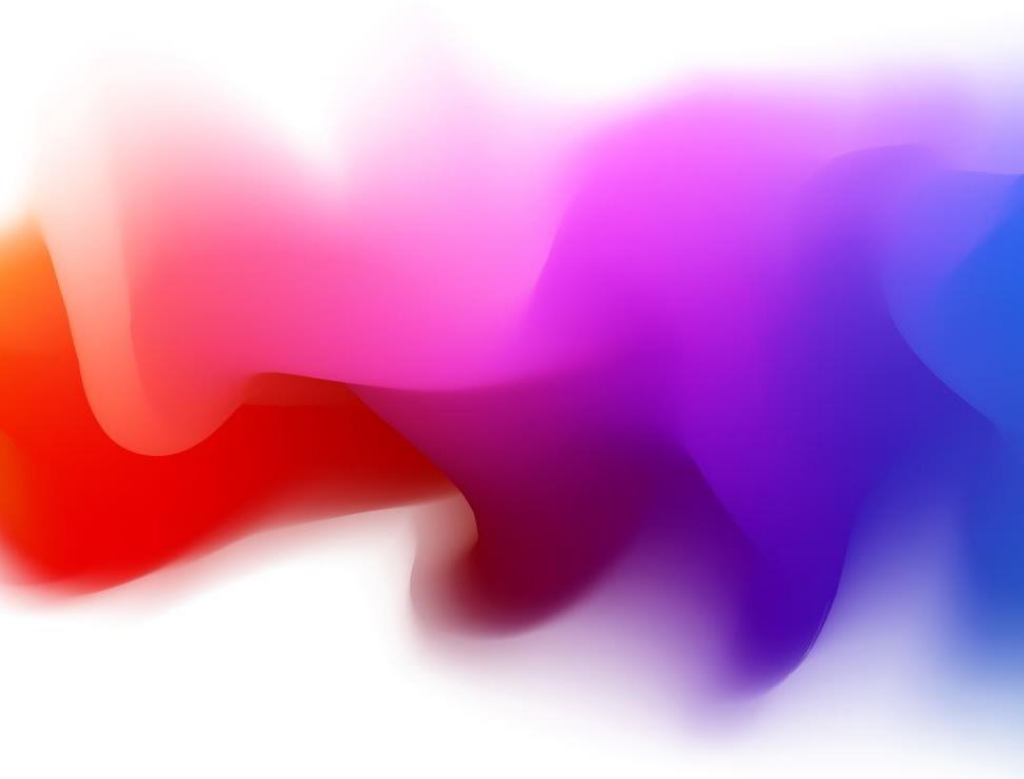
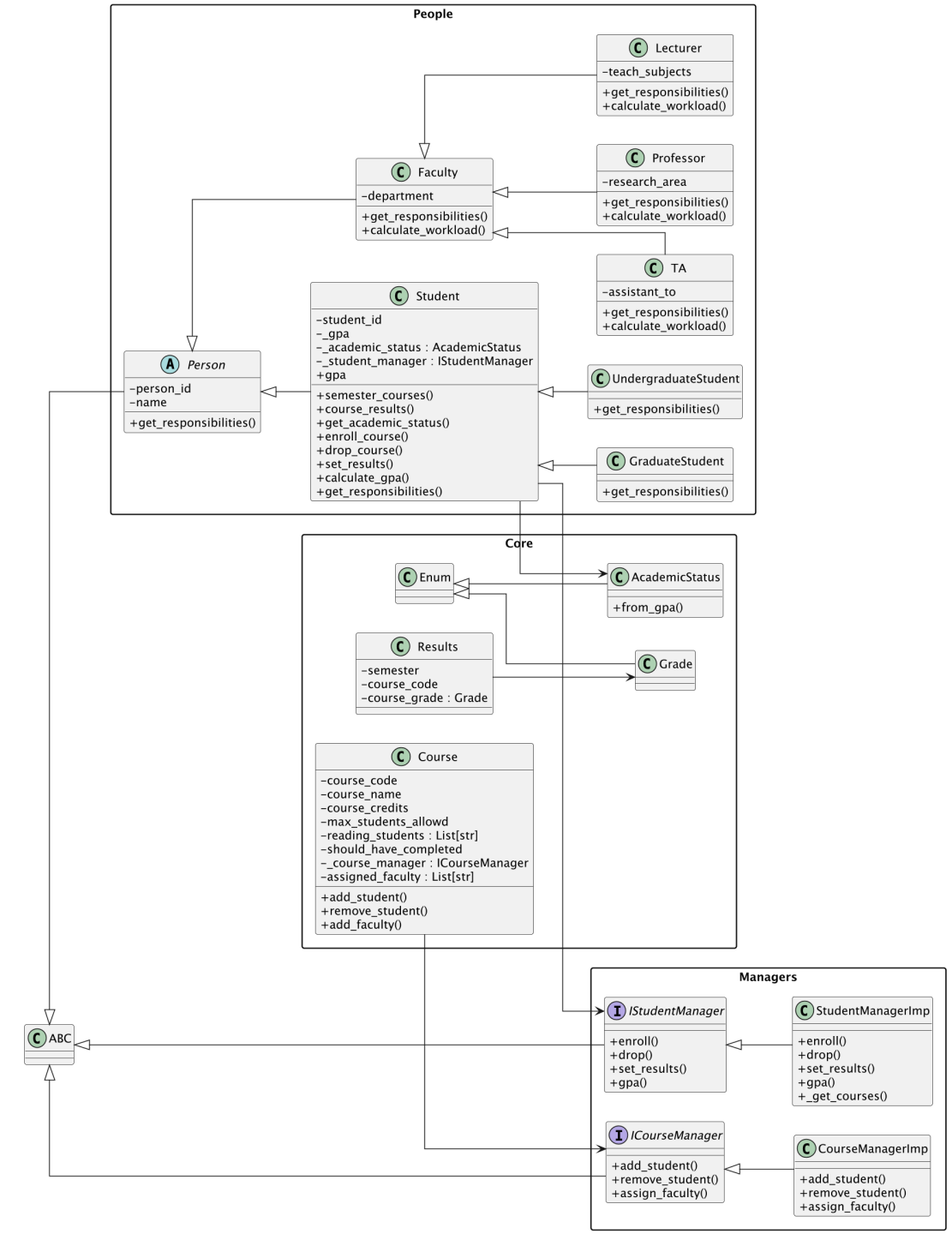




COURSE WORK: PROGRAMMING FOR DATA SCIENCE 2025.1 BATCH

COMScDS251P-015

1. UNIVERSITY SYSTEM



COURSE ENROLLMENT

- Cleaner Student class by focuss on data attributes
- Implementation with StudentManager class with Imp separation

```
from manager.student_manager import IStudentManager

class Student(Person):
    def __init__(self, person_id, name, student_manager):
        super().__init__(person_id, name)
        self.gpa = 0.0
        self._academic_status: AcademicStatus | None = None
        self._student_manager: IStudentManager | None = student_manager
        self._course_results = []

    def enroll_course(self, semester, course):
        self._student_manager.enroll(self, semester, course)

    def drop_course(self, semester, course_code):
        self._student_manager.drop(self, semester, course_code)

    def calculate_gpa(self) -> float:
        self.gpa = self._student_manager.gpa(self)
        self._academic_status = AcademicStatus.from_gpa(self.gpa)
        return self.gpa

    def get_academic_status(self) -> AcademicStatus:
        return self._academic_status
```

```
class IStudentManager(ABC):
    """Interface for student management operations."""

    @abstractmethod
    def enroll(self, student, semester, course):
        """
        Enroll a student in a specific course for a given semester.

        Args:
            student: The student object to enroll.
            semester: The semester in which the student is enrolling.
            course: The course object in which the student is enrolling.
        """
        pass

    @abstractmethod
    def drop(self, semester, course_code):
        pass

    @abstractmethod
    def gpa(self, student) -> float:
        pass
```

COURSES MANAGER

- Course Class - future proof
- Course
 - Credits
 - Max student allowed
 - Prerequisites

```
class Course:
    def __init__(self, code,
                  name,
                  credits = 3,
                  max_students_allowd = 10,
                  should_have_completed: List[str] = None,
                  course_manager: ICourseManager | None = None):
        self.course_code = code
        self.course_name = name
        self.course_credits = credits
        self.max_students_allowd = max_students_allowd
        self.reading_students: List[str] = []
        self.should_have_completed = should_have_completed
        self._course_manager: ICourseManager | None = course_manager
        self.assigned_faculty: List[str] = []

    def add_student(self, person_id: str, completed_courses: List[str]):
        self._course_manager.add_student(self, person_id, completed_courses)

    def remove_student(self, person_id: str):
        self._course_manager.remove_student(self, person_id)

    def add_faculty(self, faculty: Faculty):
        self._course_manager.assign_faculty(self, faculty)
```

```
# Course Logic interface, to separate student logics from Student model class
class ICourseManager(ABC):
    @abstractmethod
    def add_student(self, course, person_id: str, completed_courses: List[str]):
        pass

    @abstractmethod
    def remove_student(self, course, person_id: str):
        pass

    @abstractmethod
    def assign_faculty(self, course, faculty: Faculty):
        pass
```

Figure 20: Coursemanager interface and implementation at CourseMaangerImp class

COURSE ENROLLMENT AND RESULTS

```
from enum import Enum

class Grade(Enum):
    A = 4.0
    A_MINUS = 3.7
    B_PLUS = 3.3
    B = 3.0
    B_MINUS = 2.7
    C_PLUS = 2.3
    C = 2.0
    C_MINUS = 1.7
    D = 1.0
    F = 0.0
```

Figure 9: Grade Enum class

```
class Results:
    def __init__(self, semester, course_code, grade):
        self.semester = semester
        self.course_code = course_code
        self.course_grade = grade
```

Figure 10: Results class

```
class Student(Person):
    def set_results(self, results: Results):
        self._student_manager.set_results(self, results)
```

Figure 11: Assign results for course

```
class IStudentManager(ABC):
    @abstractmethod
    def set_results(self, results: Results):
        pass
```

Figure 12: Assign results signature in interface

- Results and Grade
- Validation with raise
 - Enroll - Already enrolled
 - Drop – Not Found
 - Set Results – Not Found
 - GPA – No Courses

```
class CourseManagerImp(ICourseManager):

    # Assign Student to Course
    def add_student(self, course, person_id: str, completed_courses: List[str]):
        course_code = course.course_code

        # check meets the prerequisites
        if course.should_have_completed is not None:
            for required in course.should_have_completed:
                if required not in completed_courses:
                    raise ValueError(f"Cannot enroll:{person_id} prerequisite {required} not completed for {course_code}")

        # check maximum allowed reached
        if len(course.reading_students) >= course.max_students_allowd:
            raise ValueError(f"Course {course_code} reached maximum allowed limit[{course.max_students_allowd}]")

        course.reading_students.append(person_id)
        Logger.info(f"Student {person_id} \t assigned to {course_code}")
```

COURSE MANAGER VALIDATION

2. DATA ANALYSIS: E-COMMERCE DATA ANALYSIS

```
t > requirements.txt
beautifulsoup4==4.13.5
aiohttp==3.12.15
dependency-injector==4.48.1
aiofiles==24.1.0
lxml==6.0.2
requests==2.32.5
pandas==2.3.2
matplotlib==3.10.6
seaborn==0.13.2
numpy==2.3.3
plotly==6.3.0
aioresponses==0.7.8
```


2.1 MULTI SOURCE DATA COLLECTION

```
class CollectDataUseCases:
    def __init__(
        self,
        iBookScrape: IB (parameter) iBookScrape: IBookScrape
        iEcommerceScrap iBookScrape: IBookScrape
        irssScrape: IRS iBookScrape: IBookScrape
    ):
        self.iBookScrape = iBookScrape
        self.iEcommerceScrape = iEcommerceScrape
        self.irssScrape = irssScrape

    async def booksScrape(self, pageCount: int):
        return await self.iBookScrape.collectData(pageCount)

    async def eCommerceScrape(self, pageCount: int):
        return await self.iEcommerceScrape.collectData(pageCount)

    async def rssScrape(self):
        return await self.irssScrape.collectData()
```

- Clean architecture (domain/ data)
 - Domain/UseCases
 - CollectDataUseCases [BookScrape, EcommerceScrape, RSSScrape]
 - SaveDataUseCases [SaveJson]
 - Managers
 - HttpClient, IsaveFile, IbookScrape, IECommerceScrape, IRSSXMLScrape

MULTI SOURCE DATA COLLECTION

- DI

Object	Scope
httpClient	Singleton
save_data	Singleton
webScraper	Singleton
bookScraper	Factory
ecommerceScraper	Factory
rssScraper	Factory

```
class Container(containers.DeclarativeContainer):
    config = providers.Configuration()

    use_async = os.environ.get("ASYNC", "1") == "1"

    httpClient: providers.Provider[IHttpClient] = providers.Singleton(
        HttpClientImp if use_async else HttpClientSyncImp,
        timeout=3
    )

    webScraper: providers.Provider[IWebScrape] = providers.Singleton(
        WebScrapeImp
    )

    bookScraper: providers.Provider[IBookScrape] = providers.Factory(
        BookScrapeImp,
        client=httpClient,
        scraper=webScraper
    )
```

MULTI SOURCE DATA COLLECTION

- Async Requests [coroutines/non thread blocking]
 - Blocking Request also added (enable with cmd env param)
 - Retrying failed requests
 - Exponential backoff
 - Random jitter
 - Utils
 - Logger interface with console log imp
 - Execution time
 - Unit Tests and Mocks
-

BOOK SCRAPE IMPROVEMENTS

1. Performance
2. Retry

Request Type for (books.toscrape.com)	Time Taken for 1000 books to srcape
Sync	500 + seconds
Async with descent delay	~17 seconds

2.2 DATA CLEANING AND PREPROCESSING

£ need to remove ["price", "price_excl_tax", "price_incl_tax", "tax"]

Display the first few rows of the data frame.

```
df.head()
```

	title	price	category	available	stock_count	rating	description	upc	product_type	p
0	A Light in the Attic	£51.77	Poetry	True	22	Three	It's hard to imagine a world without A Light i...	a897fe39b1053632	Books	
1	Tipping the Velvet	£53.74	Historical Fiction	True	20	One	"Erotic and absorbing...Written with starling ...	90fa61229261140a	Books	
2	Soumission	£50.10	Fiction	True	20	One	Dans une France assez proche de la nôtre, un h...	6957f44c3847a760	Books	
3	Sharp Objects	£47.82	Mystery	True	20	Four	WICKED above her hipbone, GIRL across her hear...	e00eb4fd7b871a48	Books	
4	Sapiens: A Brief History of Humankind	£54.23	History	True	20	Five	From a renowned historian comes a groundbreaki...	4165285e1663650f	Books	

Statistical Summary

```
df.describe()
```

	stock_count	num_reviews
count	1000.000000	1000.0
mean	8.585000	0.0
std	5.654622	0.0
min	1.000000	0.0
25%	3.000000	0.0
50%	7.000000	0.0
75%	14.000000	0.0
max	22.000000	0.0

No Use

2.2 DATA CLEANING AND PREPROCESSING

```
df.describe()
```

	price	stock_count	price_excl_tax	price_incl_tax	tax
count	1000.00000	1000.000000	1000.00000	1000.00000	1000.0
mean	35.07035	8.585000	35.07035	35.07035	0.0
std	14.44669	5.654622	14.44669	14.44669	0.0
min	10.00000	1.000000	10.00000	10.00000	0.0
25%	22.10750	3.000000	22.10750	22.10750	0.0
50%	35.98000	7.000000	35.98000	35.98000	0.0
75%	47.45750	14.000000	47.45750	47.45750	0.0
max	59.99000	22.000000	59.99000	59.99000	0.0

Generate

- Drop
 - price_excl_tax
 - price_incl_tax
 - tax

2.2 DATA CLEANING AND PREPROCESSING

- Text Normalization
 - Remove ...more
 - Remove leading and trailing white spaces
 - Remove tabs, consecutive white spaces with single white space
 - Drop Further columns with analysis
 - product_type which are not books are 0 – we can drop product_type column
 - Not available also 0 – we can drop available column
 - Int values for Rating, Currently string which not optimal
 - Save as Jsonl
-

2.3 ADVANCED STATISTICAL ANALYSIS

- Default has 152 books and some categories has only 1
- Health are Expensive (Avg 51.45)
- Crime are Cheaper (10.97)
- Autobiography has large price variation (std:21.47)
- Health has consistent price (2.45)
- Adult Fiction has highest rating(5) and crime has lowest(1)
- Book price gives no information about its rating, not coorelated.

2.C.1 Price distribution analysis across categories

[Generate](#) [+ Code](#) [+ Markdown](#)

```
df.groupby("category")["price"].describe()
```

	count	mean	std	min	25%	50%	75%	max
category								
Academic	1.0	13.120000	NaN	13.12	13.1200	13.120	13.1200	13.12
Add a comment	67.0	35.796418	14.193437	10.02	23.1300	36.170	48.0550	59.15
Adult Fiction	1.0	15.360000	NaN	15.36	15.3600	15.360	15.3600	15.36
Art	8.0	38.520000	12.583143	10.29	37.7175	42.080	45.2925	49.05
Autobiography	9.0	37.053333	21.469342	10.93	12.9100	40.850	57.6000	59.04

```
df.groupby("category")["rating"].mean().round(1)
```

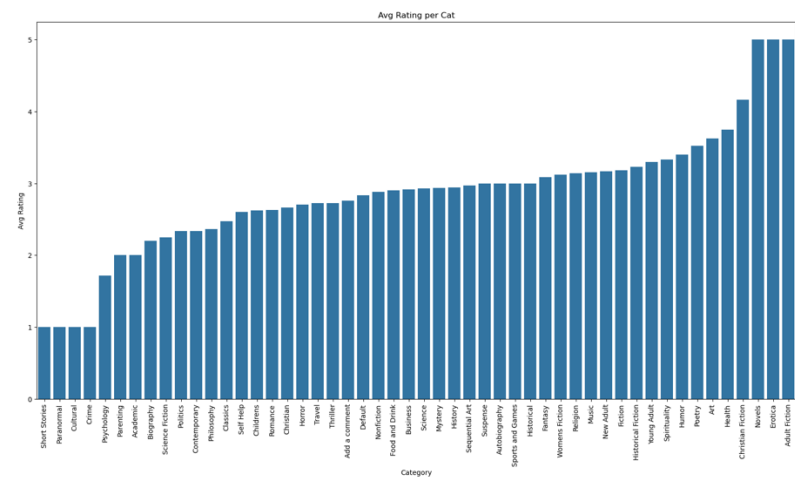
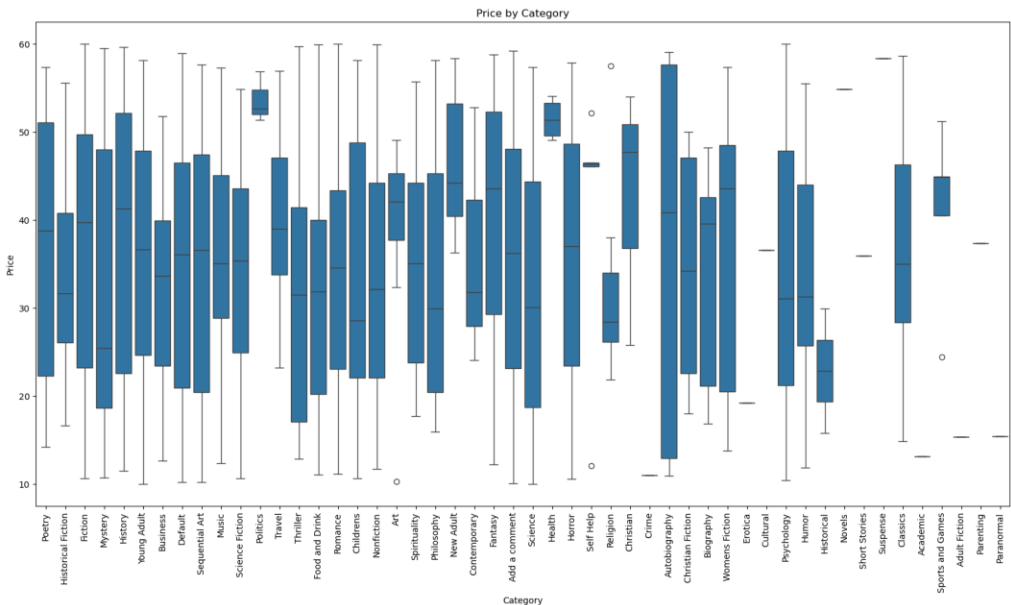
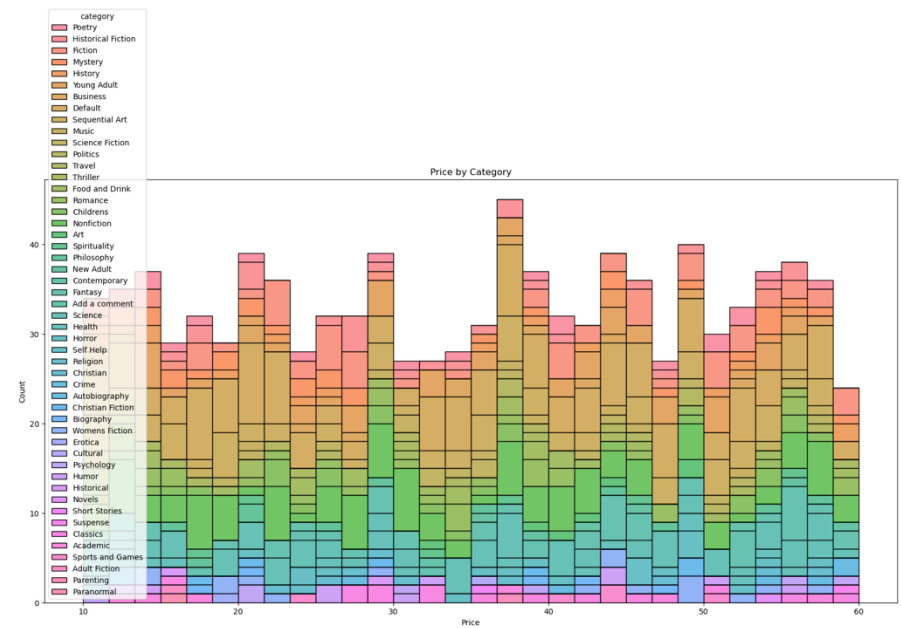
category	
Academic	2.0
Add a comment	2.8
Adult Fiction	5.0
Art	3.6

```
df[["price","rating"]].corr()
```

✓ 0.0s

	price	rating
price	1.000000	0.028166
rating	0.028166	1.000000

2.4 VISUALIZATION



PREDICTIVE ANALYSIS

- Predict pricing using the rating
 - Linear Regression
 - MSE
 - Train: 206.10
 - Test: 214.19
 - Not Overfitting
 - Weak Accuracy
-

3 DATA ETHICS: AI ETHICS IN HEALTHCARE DATA

- **Skin Cancer Detection Model**
 - Privacy Challenges - No patient identification data extracted
 - Bias
 - Demographic - 80% of images comes from male patients age over 50
 - Geographical - 100% of images comes from USA
 - Socioeconomic – 80 % of images from homeless hospitals
 - Ethical decision Making
 - Consent for using on AI
 - Secured
 - Data Audit
 - AI decision confirm from doctors
 - Carefully communicate unforeseen future risks
-

3 DATA ETHICS: AI ETHICS IN HEALTHCARE DATA

Stakeholder Impact Analysis – Case Skin Cancer

Patients

- Faster Diagnosis
 - Avoid radioactive materials
 - Risks of False detection (accuracy of the model)
 - Health Care Professionals
 - Need to train the interpreting the out put from AI
 - Trust
 - Researchers
 - Large dataset innovations but need to ensure data is used ethically
-